



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Flores Herrera Oswaldo

Semestre 2027-1

Sistemas distribuidos

Mexi Banco con Docker Compose: lo de la clase, en tu máquina.

Grupo 2.

Paso 1. Clonación del repositorio.

Para obtener el proyecto usé `git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git`, así trabajo justo con la versión que pide la práctica. Git avisó que estableciera en detached HEAD donde el repositorio apunta a una etiqueta fija y no a una rama, por lo que el código no cambia mientras hago las pruebas.0

```
C:\Users\oswal\Documents>git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
Cloning into 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Receiving objects: 100% (123/123), 46.71 KiB | 613.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
Warning: refs/tags/v05.1 cd46b8b91a09070f8262c3b8acfb17caf2405cda is not a commit!
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false
```

Paso 2. Levantamiento con Compose.

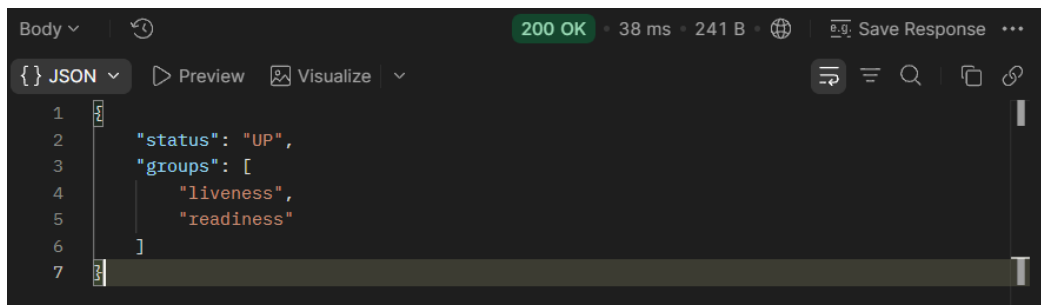
Al ejecutar `docker compose up --build -d`, se construyó la imagen del proyecto y se levantaron dos contenedores: db, correspondiente a PostgreSQL 17, y app, que contiene la aplicación desarrollada en Java. Al verificar el estado mediante `docker compose ps`, ambos servicios aparecen como healthy, confirmando que la base de datos está disponible y que la aplicación funciona correctamente. Finalmente, la aplicación queda accesible a través del puerto 8080.

```
C:\Users\oswal\Documents\mexi-banco>docker compose up --build -d
[*] Building 2 is (19/19) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 525B                                           0.0s
=> [internal] load build definition from Dockerfile                    0.0s
=> => transferring dockerfile: 457B                                     0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine 0.9s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine 0.9s
=> [internal] load .dockerignore                                       0.0s
=> => transferring context: 2B                                           0.0s
=> [builder 1/8] FROM docker.io/library/eclipse-temurin:21-jdk-alpine@sha256:0bfc69a4758a86718e5c474832d28408a8b 0.0s
=> => resolve docker.io/library/eclipse-temurin:21-jdk-alpine@sha256:0bfc69a4758a86718e5c474832d28408a8b087476 0.0s
=> [internal] load build context                                       0.0s
=> => transferring context: 3.38kB                                       0.0s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:51ab5e3302e7141ce665ca3ea85e8b5cd64 0.0s
=> => resolve docker.io/library/eclipse-temurin:21-jre-alpine@sha256:51ab5e3302e7141ce665ca3ea85e8b5cd648eafbc3c 0.0s
=> CACHED [builder 2/3] WORKDIR /app                                  0.0s
=> CACHED [builder 2/8] WORKDIR /app                                  0.0s
=> CACHED [builder 3/8] COPY pom.xml .                                0.0s
=> CACHED [builder 4/8] COPY mvnw .                                    0.0s
=> CACHED [builder 5/8] COPY .mvn .                                    0.0s
=> CACHED [builder 6/8] RUN ./mvnw dependency:go-offline -B          0.0s
=> CACHED [builder 7/8] COPY src ./src                                0.0s
=> CACHED [builder 8/8] RUN ./mvnw clean package -DskipTests         0.0s
=> CACHED [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar 0.0s
=> => exporting to image                                                0.1s
=> => exporting layers                                                 0.0s
=> => exporting manifest sha256:29d66173450432086d726c2f8e75ebc93eddc994daff89a4f60ab4a64f74fc8 0.0s
=> => exporting config sha256:b11e3bc247a98402341d0c55e4b11ee305173b87f2533b94db0c4ee3c4c5 0.0s
=> => exporting attestation manifest sha256:e6617a8c17581069cdac14da76914ef306ea50e24be68ec52c6cf4d4df62c887 0.0s
=> => exporting manifest list sha256:daad6822a5a5d2b3fcf05c48624e477a14628d9dee82f5bc486e36d211d2c941 0.0s
=> => naming to docker.io/library/mexi-banco-app:latest               0.0s
=> => unpacking to docker.io/library/mexi-banco-app:latest            0.0s
=> => resolving provenance for metadata file                           0.0s
[*] up 5/5
✔Image mexi-banco-app Built 2.3s
✔Network mexibanco Created 0.0s
✔Volume mexi-banco_mexibanco_datos Created 0.0s
✔Container mexi-banco-db-1 Healthy 0.0s
✔Container mexi-banco-app-1 Started 6.4s
```

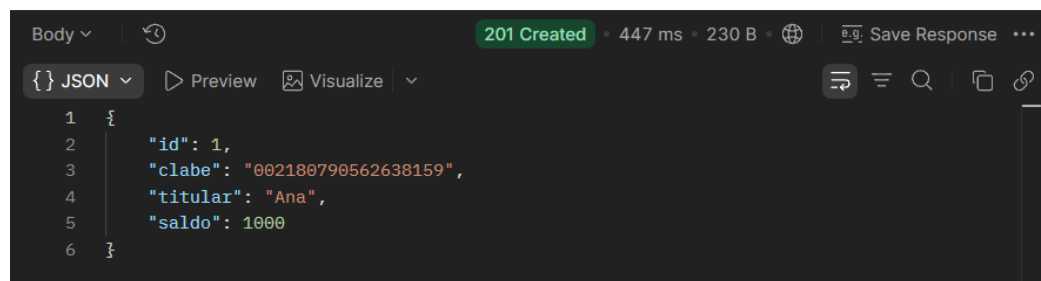
```
C:\Users\oswal\Documents\mexi-banco>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app        About a minute ago    Up 58 seconds (healthy)    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1    postgres:17-alpine  "docker-entrypoint.s..." db         About a minute ago    Up About a minute (healthy)    5432/tcp
```

Paso 3. Pruebas en Postman.

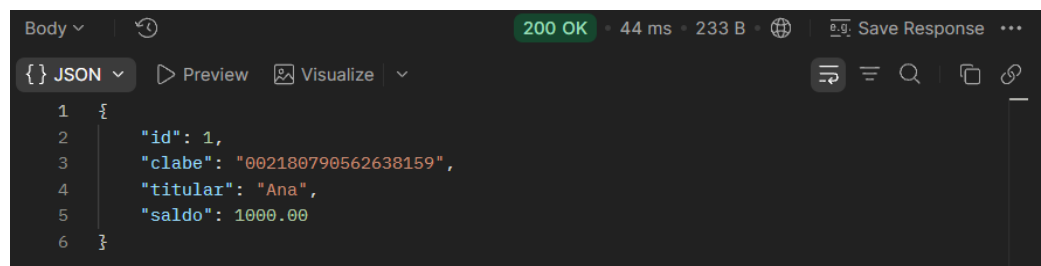
Al consultar el endpoint de salud obtuve UP con los grupos liveness y readiness. Después creé la cuenta de Ana (id 10, saldo \$1000) y la consulté para verificar que se guardó bien.



```
1 {
2   "status": "UP",
3   "groups": [
4     "liveness",
5     "readiness"
6   ]
7 }
```

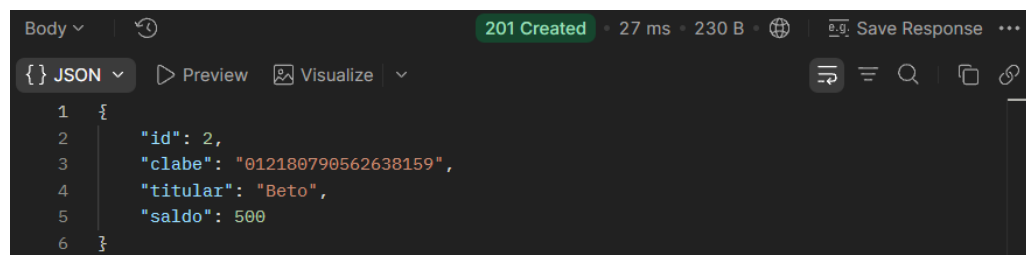


```
1 {
2   "id": 1,
3   "clabe": "002180790562638159",
4   "titular": "Ana",
5   "saldo": 1000
6 }
```



```
1 {
2   "id": 1,
3   "clabe": "002180790562638159",
4   "titular": "Ana",
5   "saldo": 1000.00
6 }
```

Luego di de alta la cuenta de Beto con \$500 y pasé \$200 de Ana a Beto. Al consultarlas, Ana quedó con \$800 y Beto con \$700, así que el monto salió de una cuenta y llegó completo a la otra.



```
1 {
2   "id": 2,
3   "clabe": "012180790562638159",
4   "titular": "Beto",
5   "saldo": 500
6 }
```

```
Body v 201 Created 80 ms 296 B e.g. Save Response ...
{} JSON Preview Visualize v
1 {
2   "id": 1,
3   "claveOrigen": "002180790562638159",
4   "claveDestino": "012180790562638159",
5   "monto": 200,
6   "fecha": "2026-09-28T02:31:38.319532770Z"
7 }
```

```
Body v 200 OK 16 ms 232 B e.g. Save Response ...
{} JSON Preview Visualize v
1 {
2   "id": 1,
3   "clabe": "002180790562638159",
4   "titular": "Ana",
5   "saldo": 800.00
6 }
```

```
Body v 200 OK 10 ms 233 B e.g. Save Response ...
{} JSON Preview Visualize v
1 {
2   "id": 2,
3   "clabe": "012180790562638159",
4   "titular": "Beto",
5   "saldo": 700.00
6 }
```

```
Body v 200 OK 33 ms 296 B e.g. Save Response ...
{} JSON Preview Visualize v
1 {
2   "id": 1,
3   "claveOrigen": "002180790562638159",
4   "claveDestino": "012180790562638159",
5   "monto": 200.00,
6   "fecha": "2026-09-28T02:31:38.319533Z"
7 }
```

Después mandé dos veces el mismo SPEI de \$50 con la misma Idempotency-Key. Las respuestas fueron idénticas (mismo id 9, estado ENVIADO) y el saldo de Ana bajó una sola vez, de \$800 a \$750. Así se ve que un reintento causado por una falla de red no genera un segundo cobro.

```
Body v 201 Created 474 ms 334 B e.g. Save Response ...
{} JSON Preview Visualize v
1 {
2   "id": 1,
3   "idempotencyKey": "prueba-790562638159",
4   "claveOrigen": "002180790562638159",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50,
8   "estado": "ENVIADO"
9 }
```

```
Body 201 Created 26 ms 337 B
{
  "id": 1,
  "idempotencyKey": "prueba-790562638159",
  "claveOrigen": "002180790562638159",
  "bancoDestino": "BBVA",
  "claveDestino": "012180000000000099",
  "monto": 50.00,
  "estado": "ENVIADO"
}
```

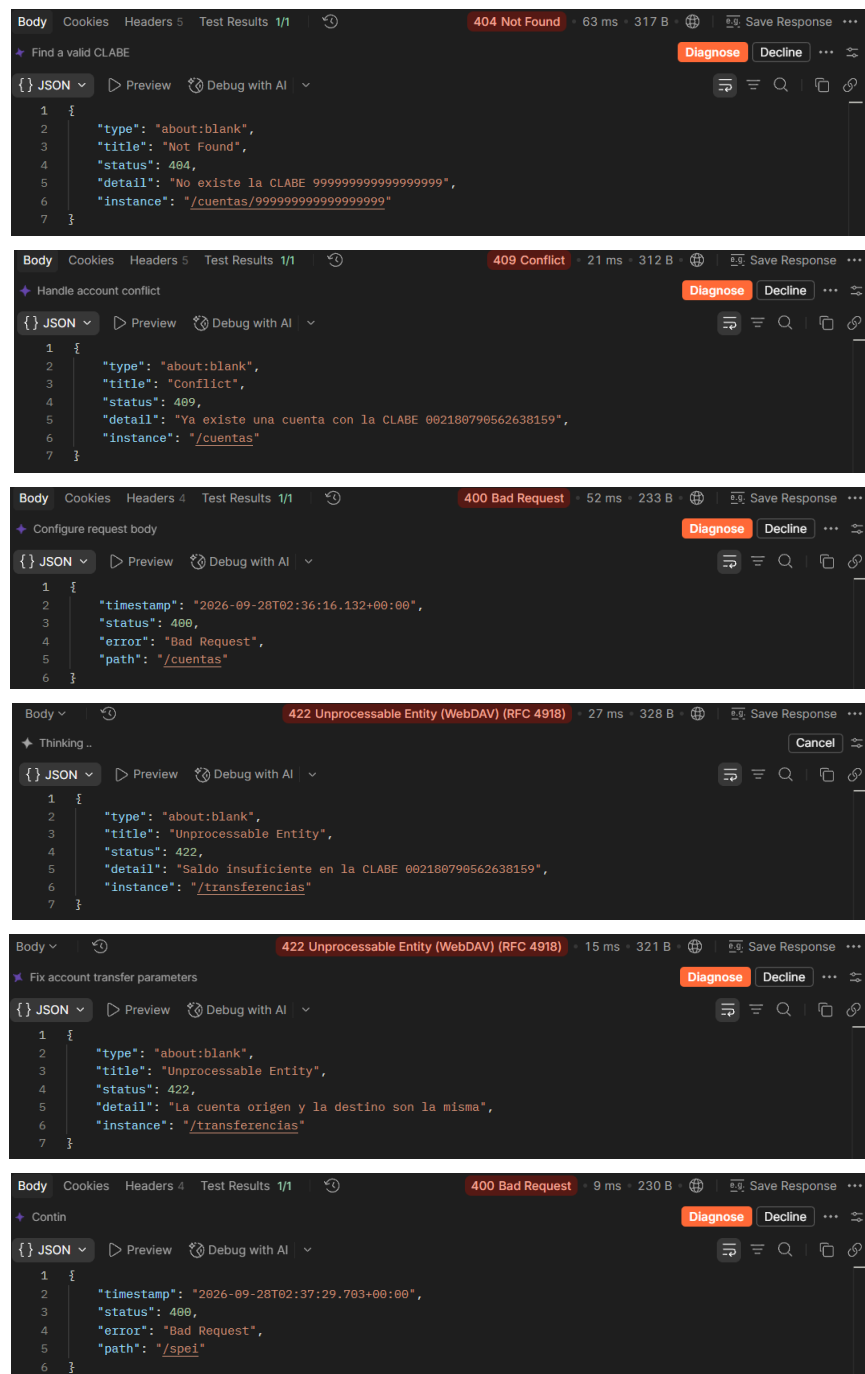
```
Body 200 OK 14 ms 232 B
{
  "id": 1,
  "clave": "002180790562638159",
  "titular": "Ana",
  "saldo": 750.00
}
```

El estado de cuenta de Ana mostró tres movimientos: el depósito inicial de 1000, la transferencia de - 200 y un solo SPEI de -\$50, para un saldo final de \$750. También quedaron dos avisos, uno por la transferencia y otro por el SPEI.

```
Body 200 OK 21 ms 712 B
[
  {
    "id": 5,
    "claveCuenta": "002180790562638159",
    "tipo": "SPEI_ENVIADO",
    "monto": -50.00,
    "saldoResultante": 750.00,
    "fecha": "2026-09-28T02:32:49.977353Z",
    "referencia": "BBVA - 012180000000000099"
  },
  {
    "id": 3,
    "claveCuenta": "002180790562638159",
    "tipo": "TRANSFERENCIA_ENVIADA",
    "monto": -200.00,
    "saldoResultante": 800.00,
    "fecha": "2026-09-28T02:31:38.314216Z",
    "referencia": "a 012180790562638159"
  },
  {
    "id": 1,
    "claveCuenta": "002180790562638159",
    "tipo": "DEPOSITO",
    "monto": 1000.00,
    "saldoResultante": 1000.00,
    "fecha": "2026-09-28T02:30:38.708768Z",
    "referencia": "saldo inicial"
  }
]
```

```
Body 200 OK 22 ms 453 B
[
  {
    "id": 3,
    "claveCuenta": "002180790562638159",
    "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
    "fecha": "2026-09-28T02:32:50.393240Z"
  },
  {
    "id": 1,
    "claveCuenta": "002180790562638159",
    "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
    "fecha": "2026-09-28T02:31:38.325024Z"
  }
]
```

Para terminar, implementé fallos a propósito con las peticiones de error que trae la colección y revisé qué respondía la API en cada caso. Pedir los datos de una CLAVE que nadie ha registrado devolvió un 404 Not Found. Intentar dar de alta otra cuenta con una CLAVE que ya estaba en uso terminó en un 409 Conflict. Cuando ordené una transferencia por un monto mayor al que había en la cuenta, o con la misma cuenta como origen y destino, la respuesta fue un 422 Unprocessable Entity. Y al enviar una petición a la que le faltaba algún campo, el resultado fue un 400 Bad Request. Después de todas estas pruebas volví a mirar la cuenta y el saldo seguía igual que antes.



```
Body Cookies Headers 5 Test Results 2/2 200 OK 19 ms 232 B Save Response ...
{ } JSON Preview Visualize
1 {
2   "id": 1,
3   "clabe": "002180790562638159",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

Paso 4. Bajar y subir volumen

Con docker compose down se borraron los contenedores y la red, pero el volumen quedó intacto. Al volver a levantar el entorno, Ana seguía con \$750, o sea que su información no dependía de los contenedores.

```
C:\Users\oswal\Documents\mexi-banco>docker compose down
[+] down 3/3
✓Container mexi-banco-app-1 Removed 0.5s
✓Container mexi-banco-db-1 Removed 0.2s
✓Network mexibanco Removed 0.2s
C:\Users\oswal\Documents\mexi-banco>docker compose up -d
[+] up 3/3
✓Network mexibanco Created 0.0s
✓Container mexi-banco-db-1 Healthy 5.9s
✓Container mexi-banco-app-1 Started 6.0s
```

```
Body Cookies Headers 5 Test Results 2/2 200 OK 245 ms 232 B Save Response ...
{ } JSON Preview Visualize
1 {
2   "id": 1,
3   "clabe": "002180790562638159",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

Con docker compose down -v también se eliminó el volumen, y la misma consulta devolvió 404: la base arrancó vacía.

```
C:\Users\oswal\Documents\mexi-banco>docker compose down -v
[+] down 4/4
✓Container mexi-banco-app-1 Removed 0.4s
✓Container mexi-banco-db-1 Removed 0.3s
✓Volume mexi-banco_mexibanco_datos Removed 0.0s
✓Network mexibanco Removed 0.2s
C:\Users\oswal\Documents\mexi-banco>docker compose up -d
[+] up 4/4
✓Network mexibanco Created 0.0s
✓Volume mexi-banco_mexibanco_datos Created 0.0s
✓Container mexi-banco-db-1 Healthy 5.9s
✓Container mexi-banco-app-1 Started 6.1s
```

```
Body Cookies Headers 5 Test Results 0/2 404 Not Found 231 ms 317 B Save Response ...
{ } JSON Preview Debug with AI
1 {
2   "type": "about:blank",
3   "title": "Not Found",
4   "status": 404,
5   "detail": "No existe la CLABE 002180790562638159",
6   "instance": "/cuentas/002180790562638159"
7 }
```

La conclusión es que los datos están en el volumen y no en el contenedor. Los contenedores se pueden destruir y recrear cuando sea, pero el volumen es lo que hay que proteger, porque ahí guarda PostgreSQL sus archivos y solo se pierden si ese volumen se borra.

Paso 5. Revisión del código.

El proyecto se divide en cinco módulos: cuenta, movimiento, transferencia, notificación y spei. Cada uno tiene cuatro capas: controlador que es el que traduce las peticiones HTTP, servicio que contiene las reglas de negocio y transacciones, repositorio y entidad. Un módulo nunca accede al repositorio de otro; cuando necesita algo ajeno, se lo pide al servicio correspondiente.

La idempotencia del SPEI se resuelve guardando cada solicitud junto con su Idempotency-Key en una columna UNIQUE. Si el INSERT falla por duplicado, el sistema devuelve la solicitud ya registrada en lugar de cobrar otra vez, y así un reintento no genera un segundo cargo.

Paso 6. Compose y Dockerfile.

docker-compose.yml. Declara dos servicios, la red mexibanco y el volumen mexibanco_datos. db usa postgres:17-alpine, guarda los datos en el volumen y comprueba su estado con pg_isready. app se construye con el Dockerfile, publica el 8080 y encuentra la base por el nombre del servicio (DB_HOST: db), sin IPs. Solo arranca cuando la base está healthy, su healthcheck consulta /actuador/health. La configuración viaja en variables de entorno; las credenciales están a la vista a propósito porque son solo de desarrollo local.

Dockerfile. Tiene dos etapas. En la primera, con JDK 21, se copia el pom.xml y se bajan las dependencias antes de traer el código, y luego se compila el .jar. En la segunda, con solo JRE 21, se copia el .jar y se ejecuta. Así la imagen final es más chica, porque no lleva Maven, JDK ni código fuente.